

Práctica 1: Cálculo de eficiencia de algoritmos

Grado en Ingeniería Informática

Algorítmica · Subgrupo A2

Autores*:

López Lozano, Francisco Javier

Martos Molina, Enrique

Mestre Redondo, Iván

Molina Cobo, Juan Ignacio

Marzo de 2026

1. Introducción

En esta práctica se analiza el comportamiento empírico de nueve algoritmos clásicos: los algoritmos de ordenación por inserción, selección, burbuja y shellsort, de eficiencia teórica $O(n^2)$; mergesort y heapsort, de eficiencia $O(n \log n)$; quicksort, con eficiencia media $\Theta(n \log n)$ y peor caso $O(n^2)$; el algoritmo recursivo de Fibonacci, de orden $O(\varphi^n)$ con $\varphi \approx 1,618$; y el algoritmo de las Torres de Hanoi, de orden $O(2^n)$.

Para cada algoritmo se han medido los tiempos de ejecución sobre distintos tamaños de entrada, se han generado ajustes híbridos al término dominante de su eficiencia teórica y se han comparado los resultados entre algoritmos de mismo orden.

Todas las mediciones se han realizado en el siguiente entorno:

- **Procesador:** Apple M2, 8 núcleos, 3,5 GHz, arquitectura ARM
- **Memoria:** 8 GB (caché L1 64 KB, L2 4 MB)
- **Sistema operativo:** macOS 26.3.1 Tahoe
- **Compilador:** g++ -std=c++11 -O0

Nota: esta memoria, así como los códigos fuente y scripts empleados para recrear los resultados, está disponible en <https://github.com/igmc/alg-p1-eficiencia>.

*Porcentaje de participación de cada integrante: 25 % cada uno. En comentarios, cada archivo del proyecto indica su autoría.

2. Eficiencia empírica

Metodología: Los tiempos de ejecución han sido recogidos mediante un script `shell` que lanza cada algoritmo sobre vectores de tamaño creciente y registra el tiempo de CPU empleado. Para cada tamaño de entrada se ha realizado una única medición. Los algoritmos de distinto orden de eficiencia se han ejecutado en tablas separadas dado que sus rangos de n difieren significativamente.

2.1. Algoritmos cuadráticos

El Cuadro 1 recoge los tiempos de ejecución de los algoritmos de orden cuadrático para tamaños de entrada entre 8000 y 200000.

n	Burbuja	Insercion	Seleccion	Shellsort	SS peor	QS peor
8000	0,087	0,056	0,091	0,004	0,001	0,109
16000	0,428	0,228	0,324	0,004	0,001	0,428
24000	1,026	0,460	0,726	0,006	0,002	0,958
32000	2,194	0,803	1,277	0,009	0,002	1,772
40000	3,266	1,256	2,053	0,012	0,002	2,703
48000	5,048	1,804	2,960	0,016	0,003	3,913
56000	6,628	2,623	3,847	0,014	0,003	5,211
64000	8,109	3,179	5,058	0,018	0,004	6,922
72000	10,403	4,097	6,342	0,019	0,005	8,473
80000	12,732	4,925	7,909	0,022	0,005	10,421
88000	15,305	5,981	9,542	0,024	0,006	12,638
96000	18,266	7,100	11,268	0,028	0,006	15,041
104000	21,347	8,324	13,179	0,029	0,007	17,656
112000	24,629	9,628	15,250	0,032	0,007	20,369
120000	28,422	11,223	17,644	0,034	0,008	23,440
128000	32,458	12,615	19,827	0,043	0,008	26,624
136000	36,493	14,187	22,424	0,041	0,009	29,990
144000	40,902	16,054	25,424	0,041	0,009	33,720
152000	44,114	17,702	28,152	0,044	0,010	37,565
160000	50,543	19,683	31,134	0,050	0,011	41,526
168000	55,578	21,652	34,114	0,051	0,011	45,833
176000	61,259	23,630	37,413	0,053	0,012	
184000	66,893	25,861	41,063	0,057	0,013	
192000	72,853	28,223	44,465	0,063	0,013	
200000	79,143	30,613	48,433	0,063	0,014	

Cuadro 1: tiempos de ejecución (s) de los algoritmos de orden $O(n^2)$.

La Figura 1 muestra la comparativa gráfica de los cinco algoritmos. Los tiempos de inserción, selección y burbuja presentan la pendiente característica de orden 2 en escala log-log, en concordancia con su eficiencia teórica.

Se incluye asimismo el peor caso de quicksort en esta tabla, pese a que su eficiencia en caso promedio es $\Theta(n \log n)$. Esta decisión se justifica porque el peor caso de quicksort

es $O(n^2)$, por lo que su comportamiento en dicho caso es comparable al del resto de algoritmos cuadráticos y resulta más informativo incluirlo aquí que en la tabla loglineal.

Shellsort, pese a ser teóricamente $O(n^2)$, muestra un comportamiento empírico considerablemente mejor, próximo a $O(n^{1.5})$ o $O(n \log^2 n)$ según la secuencia de gaps empleada. Esto explica el mal ajuste cuadrático observado. La Figura 2 compara dicho ajuste con uno de la forma $f(x) = a \cdot x^\alpha$, obteniendo $\alpha = 1,1106$.

2.2. Algoritmos loglineales

El Cuadro 2 recoge los tiempos de ejecución de los algoritmos de orden $O(n \log n)$ para tamaños de entrada entre 500000 y 5000000.

La Figura 3 muestra la comparativa gráfica de los cuatro algoritmos. En condiciones promedio, mergesort es el más eficiente en tiempo, seguido de quicksort y heapsort en un rango próximo entre sí. Destaca, sin embargo, la diferencia entre el caso promedio y el peor caso de mergesort, que se separa notablemente del resto de series. No se han medido los peores casos de quicksort y heapsort, por lo que no es posible establecer una comparativa completa entre casos extremos.

2.3. Algoritmos exponenciales

El Cuadro 3 recoge los tiempos de ejecución de los algoritmos de orden exponencial para tamaños de entrada entre 3 y 33.

La Figura 4 muestra la comparativa gráfica de ambos algoritmos en escala log-lineal. Los puntos empíricos se alinean con precisión sobre la curva ajustada para valores de n suficientemente grandes, confirmando el crecimiento exponencial predicho por la teoría. Hanoi crece sensiblemente más rápido que Fibonacci, diferencia que refleja directamente la distancia entre las bases 2 y $\varphi \approx 1,618$. Para n pequeño se observa mayor dispersión respecto al ajuste, comportamiento esperable dado que el tiempo de ejecución está dominado por factores constantes del sistema más que por el término dominante.

3. Cálculo híbrido de la eficiencia

Para cada algoritmo se ha realizado un ajuste híbrido al término dominante de su eficiencia teórica, obteniendo una función de la forma $f(x) = a \cdot g(x)$ donde $g(x)$ es el término dominante y a es el coeficiente ajustado. Los ajustes se han realizado con gnuplot sobre los mismos datos empíricos de la sección anterior.

3.1. Algoritmos cuadráticos

Para los algoritmos de orden $O(n^2)$ se ha ajustado la función $f(x) = a \cdot x^2$. Los coeficientes obtenidos son los siguientes:

- Inserción: $a = 7,66757 \times 10^{-10}$
- Selección: $a = 1,21257 \times 10^{-9}$
- Burbuja: $a = 1,97198 \times 10^{-9}$

Algoritmos Cuadráticos

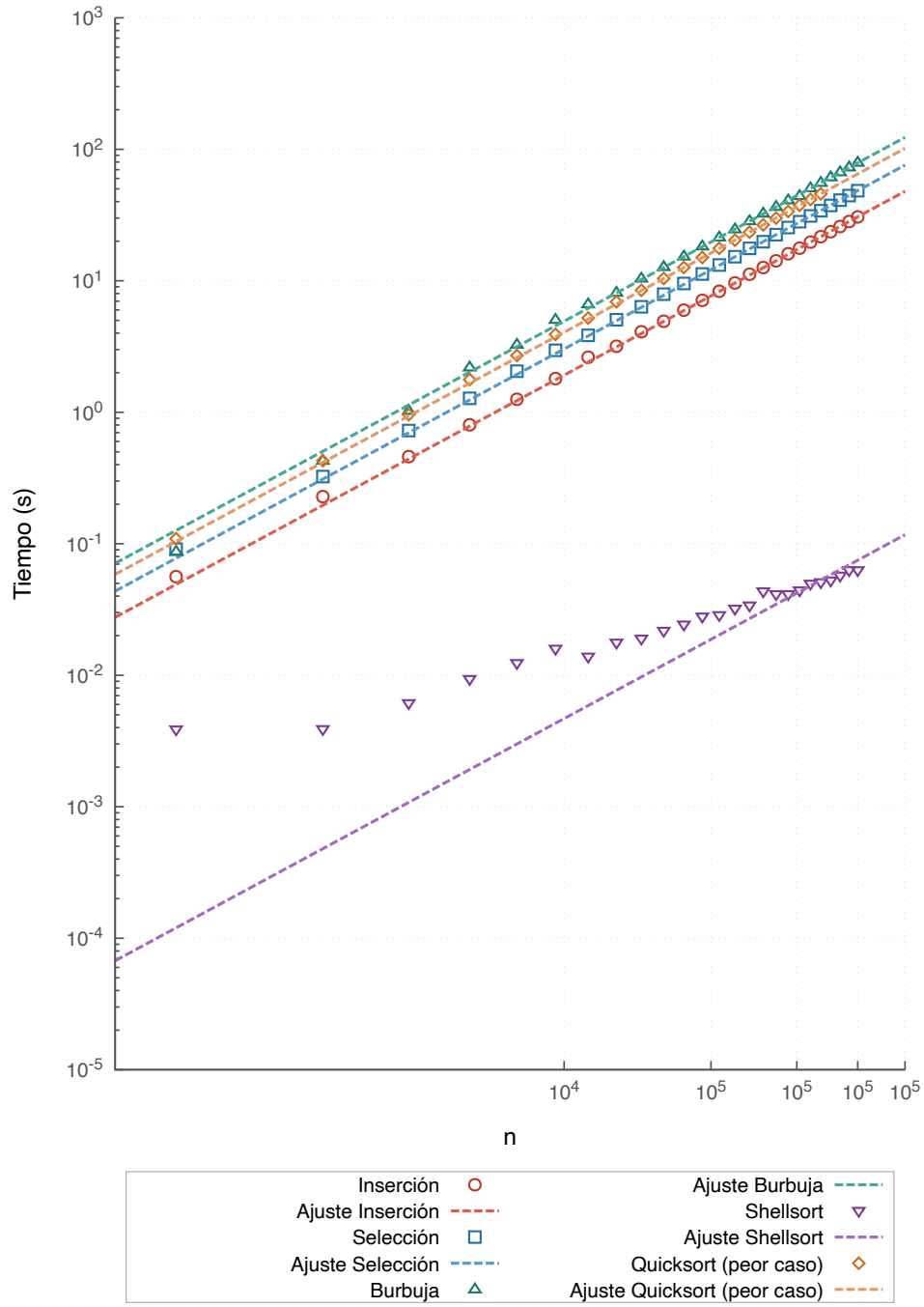


Figura 1: Tiempos de ejecución de los algoritmos de orden $O(n^2)$ (escala log-log).

Comparativa de ajustes para shellsort

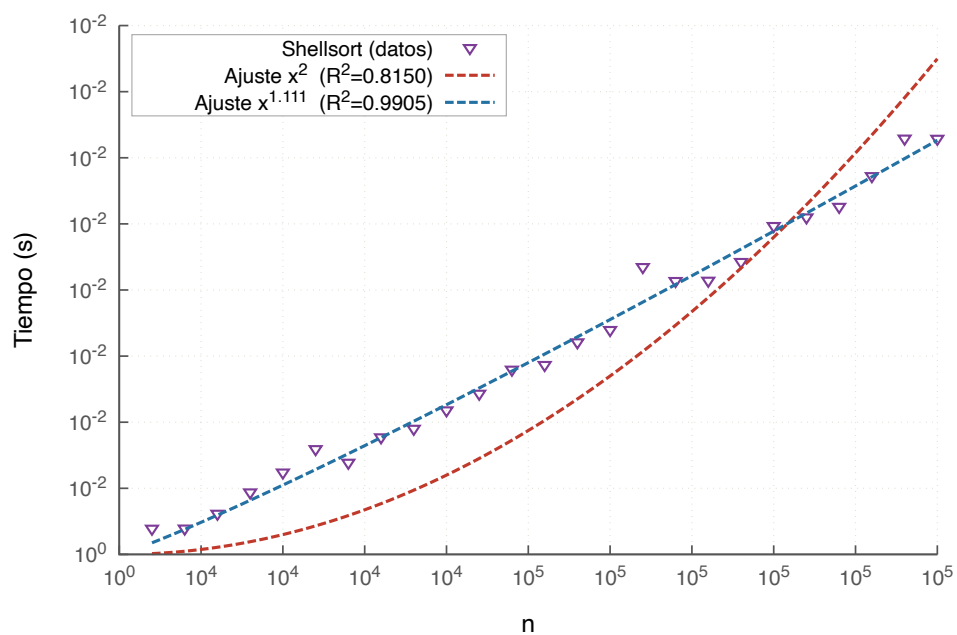


Figura 2: Comparación de ajustes para shellsort (escala lineal).

Algoritmos log-lineales

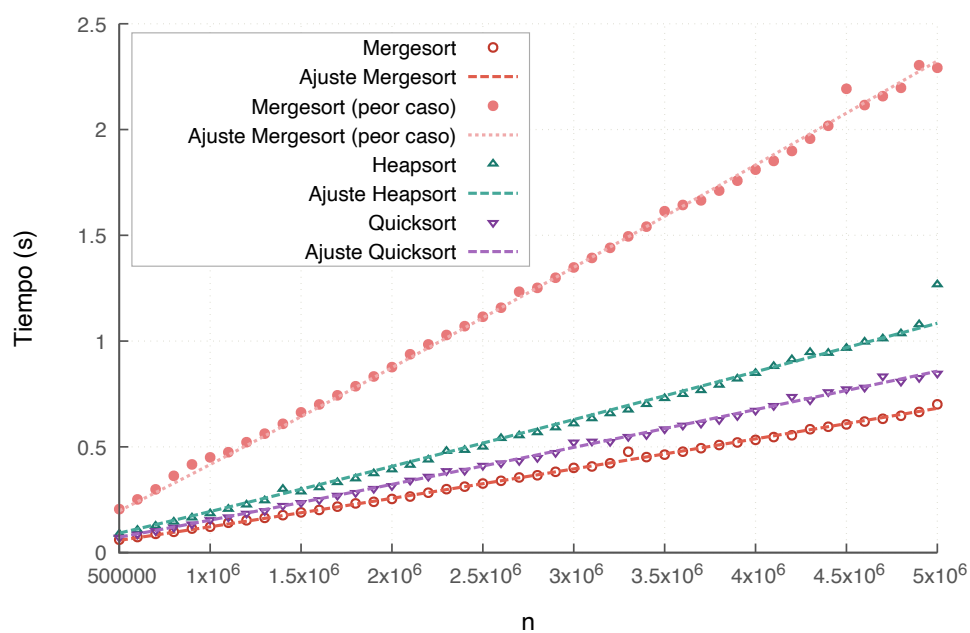


Figura 3: Tiempos de ejecución de los algoritmos de orden $O(n \log n)$ (escala lineal).

n	Mergesort	Heapsort	Heapsort (peor caso)	Quicksort
100000	0,015	0,087	0,118	0,073
200000	0,025	0,105	0,140	0,088
300000	0,035	0,125	0,175	0,104
400000	0,047	0,144	0,205	0,120
500000	0,057	0,164	0,222	0,136
600000	0,071	0,183	0,248	0,156
700000	0,089	0,205	0,270	0,169
800000	0,099	0,225	0,314	0,186
900000	0,110	0,245	0,345	0,199
1000000	0,122	0,299	0,356	0,223
1100000	0,137	0,287	0,395	0,237
1200000	0,147	0,307	0,430	0,250
1300000	0,165	0,331	0,436	0,271
1400000	0,188	0,348	0,462	0,284
1500000	0,202	0,373	0,490	0,304
1600000	0,213	0,391	0,518	0,317
1700000	0,235	0,413	0,548	0,343
1800000	0,277	0,437	0,575	0,361
1900000	0,275	0,478	0,605	0,388
2000000	0,268	0,483	0,639	0,389
2100000	0,274	0,499	0,661	0,413
2200000	0,295	0,539	0,685	0,424
2300000	0,313	0,552	0,717	0,436
2400000	0,328	0,566	0,737	0,450
2500000	0,341	0,590	0,783	0,474
2600000	0,354	0,608	0,825	0,523
2700000	0,370	0,633	0,853	0,527
2800000	0,387	0,656	0,899	0,526
2900000	0,418	0,674	0,908	0,549
3000000	0,419	0,700	0,934	0,558
3100000	0,444	0,728	1,003	0,585
3200000	0,497	0,747	0,999	0,603
3300000	0,464	0,765	1,027	0,612
3400000	0,480	0,791	1,195	0,629
3500000	0,494	0,820	1,315	0,648
3600000	0,501	0,847	1,134	0,673
3700000	0,515	0,880	1,144	0,696
3800000	0,535	0,912	1,224	0,738
3900000	0,554	0,946	1,217	0,722
4000000	0,566	0,942	1,277	0,761
4100000	0,566	0,965	1,297	0,775
4200000	0,576	0,995	1,374	0,782
4300000	0,595	1,010	1,381	0,835
4400000	0,618	1,034	1,377	0,810
4500000	0,639	1,077	1,396	0,828
4600000	0,664	1,266	1,423	0,848
4700000	0,670			
4800000	0,683			
4900000	0,686			
5000000	0,699			

n	Fibonacci	Hanoi
3	0,00000	0,00001
4	0,00000	0,00001
5	0,00000	0,00000
6	0,00000	0,00001
7	0,00000	0,00001
8	0,00000	0,00001
9	0,00000	0,00003
10	0,00000	0,00005
11	0,00000	0,00010
12	0,00000	0,00019
13	0,00000	0,00039
14	0,00000	0,00078
15	0,00000	0,00156
16	0,00001	0,00308
17	0,00001	0,00630
18	0,00002	0,01269
19	0,00003	0,02535
20	0,00004	0,05033
21	0,00007	0,10052
22	0,00010	0,20251
23	0,00017	0,40697
24	0,00028	0,82528
25	0,00044	1,61193
26	0,00072	3,71528
27	0,00116	6,52792
28	0,00196	12,80170
29	0,00300	25,65850
30	0,00487	50,44730
31	0,00789	100,78100
32	0,01277	200,78100
33	0,02073	406,43100
34	0,03284	
35	0,05296	

Cuadro 3: tiempos de ejecución (s) de los algoritmos de orden $O(c^2)$, $c > 1$.

Algoritmos Exponenciales

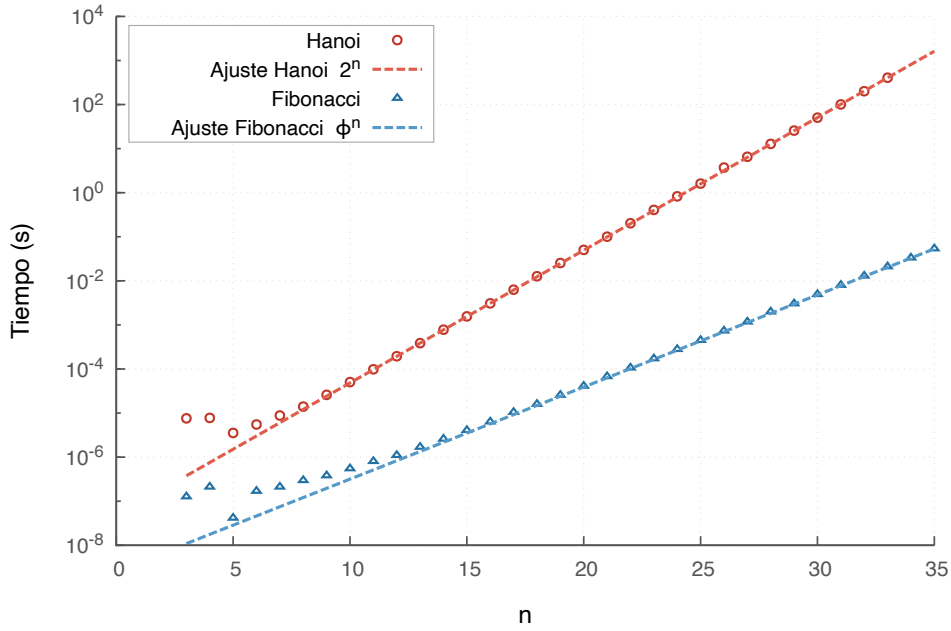


Figura 4: Tiempos de ejecución de los algoritmos de orden exponencial (escala log-lineal).

- Shellsort: $a = 1,87424 \times 10^{-12}$
- Quicksort (peor caso): $a = 1,62547 \times 10^{-9}$

Los coeficientes de inserción, selección y burbuja son del mismo orden de magnitud, lo que es consistente con su comportamiento similar observado en la sección anterior. El coeficiente de quicksort en peor caso es comparable al de selección y burbuja, confirmando que su crecimiento cuadrático en dicho caso es real.

El caso de shellsort es nuevamente destacable: el coeficiente ajustado $a = 1,87424 \times 10^{-12}$ es varios órdenes de magnitud menor que el del resto, reflejo directo de que la función x^2 sobreestima su crecimiento real. La Figura 2, mostrada anteriormente en la sección de eficiencia empírica, ilustra esta discrepancia comparando el ajuste cuadrático con un ajuste libre de la forma $f(x) = a \cdot x^\alpha$, que produce $\alpha = 1,1106$, valor próximo a 1,2 y alejado del exponente teórico 2. La mejora en la bondad del ajuste es notable: el coeficiente de determinación pasa de $R^2 = 0,814991$ con el ajuste cuadrático a $R^2 = 0,990489$ con el ajuste libre.

3.2. Algoritmos loglineales

Para los algoritmos de orden $O(n \log n)$ se ha ajustado la función $f(x) = a \cdot x \log x$. Los coeficientes obtenidos son:

- Mergesort: $a = 8,84509 \times 10^{-9}$
- Mergesort (peor caso): $a = 3,01384 \times 10^{-8}$
- Heapsort: $a = 1,40579 \times 10^{-8}$

- Quicksort (promedio): $a = 1,11192 \times 10^{-8}$

El coeficiente de mergesort en caso promedio es el menor del grupo, en concordancia con ser el más rápido en condiciones normales. El coeficiente de su peor caso es aproximadamente tres veces mayor, lo que cuantifica la degradación observada en la sección anterior. Heapsort y quicksort presentan coeficientes intermedios y próximos entre sí.

3.3. Algoritmos exponenciales

Para Hanoi y Fibonacci se han ajustado las funciones $f(x) = a \cdot 2^x$ y $f(x) = a \cdot \varphi^x$ respectivamente, con $\varphi \approx 1,618$. Los coeficientes obtenidos son:

- Hanoi: $a = 4,71889 \times 10^{-8}$
- Fibonacci: $a = 2,57762 \times 10^{-9}$

Ambos ajustes presentan una bondad elevada, como se aprecia en la Figura 4, donde los puntos empíricos siguen con precisión las curvas ajustadas para valores de n suficientemente grandes.

4. Comparativa global de ordenación

La Figura 5 muestra los tiempos de ejecución de todos los algoritmos de ordenación sobre un rango común de tamaños de entrada entre 8000 y 200000, permitiendo apreciar directamente las diferencias entre órdenes de eficiencia.

La separación entre algoritmos $O(n^2)$ y $O(n \log n)$ es clara: mergesort, heapsort y quicksort en caso promedio presentan tiempos significativamente menores que los algoritmos cuadráticos en todo el rango, con inserción, selección y burbuja creciendo varios órdenes de magnitud por encima. Esto ilustra de forma empírica la ventaja práctica de los algoritmos loglineales frente a los cuadráticos a partir de tamaños de entrada moderados.

Los peores casos de quicksort y heapsort muestran una degradación notable respecto a sus casos promedio, llegando a superar a los algoritmos cuadráticos para n suficientemente grande. Así, queda de manifiesto que la eficiencia teórica en caso promedio no garantiza un buen rendimiento en todos los escenarios.

Shellsort presenta tiempos muy reducidos en ambas series: caso promedio y peor caso. Esto confirma lo discutido en la sección anterior; el vector en orden decreciente no constituye el peor caso real para la secuencia de gaps empleada, y el algoritmo lo resuelve de forma casi óptima.

5. Conclusiones

Los resultados obtenidos confirman en términos generales las eficiencias teóricas de los algoritmos estudiados. La separación entre órdenes de eficiencia es clara y consistente. Los algoritmos $O(n \log n)$ superan ampliamente a los cuadráticos a partir de tamaños de entrada moderados, y los exponenciales escalan de forma inviable en la práctica; Hanoi supera los 400 segundos para $n = 33$.

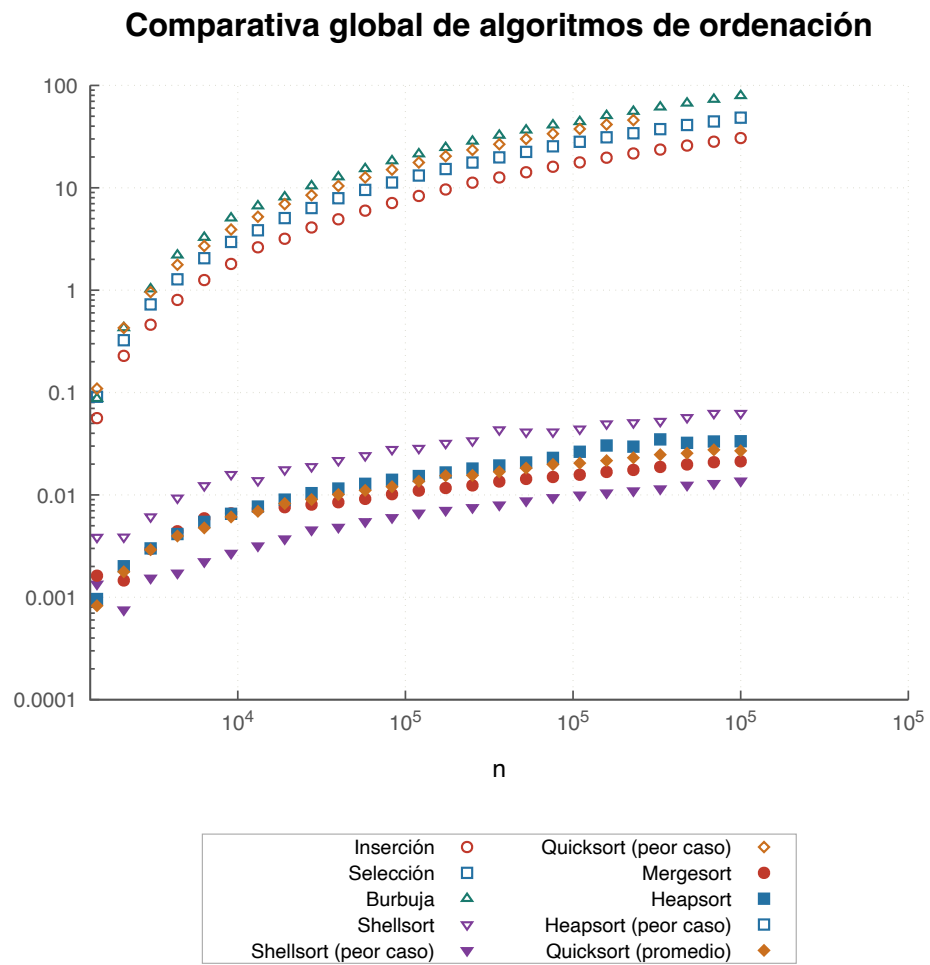


Figura 5: Comparativa global de algoritmos de ordenación (escala log-lineal).

Entre los algoritmos de ordenación, mergesort presenta el mejor rendimiento en caso promedio, si bien su peor caso muestra una degradación notable. Quicksort y heapsort ofrecen un rendimiento intermedio y más uniforme, aunque no se han medido sus peores casos en este estudio. Los algoritmos cuadráticos — inserción, selección y burbuja — presentan tiempos similares entre sí, siendo inserción ligeramente más rápido en la práctica.

El caso más destacable es shellsort, cuyo comportamiento empírico se aleja significativamente de su eficiencia teórica $O(n^2)$. El ajuste libre sobre los datos empíricos produce un exponente $\alpha = 1,1106$, muy inferior al teórico, lo que pone de manifiesto que la secuencia de gaps empleada determina en gran medida el rendimiento real del algoritmo. Asimismo, el vector en orden decreciente, empleado en esta práctica como peor caso, no logra degradar el rendimiento.

En cuanto a los algoritmos exponenciales, ambos ajustes presentan una bondad elevada, confirmando el crecimiento predicho por la teoría. La diferencia entre las bases 2 y $\varphi \approx 1,618$ se traduce en una separación de varios órdenes de magnitud entre Hanoi y Fibonacci para n moderado.